

Microprocessor Simulators

Indispensable for Embedded System Developers

This month's EFY-CD includes some of the simulation software that support microprocessors from vendors like Atmel and Renesas

■ ABHIJIT PAUL CHOUDHURY

The work of a microprocessor can be divided into operations done inside the microprocessor (such as addition, subtraction and logical operations), operations done external to the processor (reading/writing to the memory or to the peripherals) and operations initiated by the slaves and peripherals.

But, why does a developer need a microprocessor simulator? A simulator, as the name implies, helps in code development and debugging in the absence of actual hardware, by simulating the microprocessor. Rizwan Masharib, area sales manager-Southwest Asia, IAR Systems, says, "Microprocessors need instructions in a low-level language to perform specific functionalities. It is difficult to develop complex systems in this low-level language. Compilers perform the task of efficiently and unambiguously translating high-level code such as C/C++ into machine-readable instructions (Assembly language)."

He adds, "Compiler, simulator and debugger are typically available as single application software, termed as integrated development environment (IDE)."

Derek Carlson, vice president of Microchip Technology's Development Tools group, says, "Simulator lets you develop code prior to obtaining the hardware. It contains very powerful stimulus, which simulates the hardware. Additionally, it allows you to try new devices or alternatives that might suit the project better, without waiting on the silicon samples. Finally, it can be used to provide testing that is impossible to do in hardware, such as simultaneous interrupts."

Do your homework

Before buying any microprocessor software, there are a number of features that you must look for to get the most out of it. Masharib says, "Support, open architecture, code optimisation and market reputation are a few of the important features to look out for." Further, he says, check how good is

technical support for the toolchain. How well does it integrate with other components such as third-party stacks, RTOS and debuggers? How compact and robust is the code produced? In the next five years, will the toolchain vendor still be supporting this MCU?



IAR Embedded Workbench is a development tool for embedded applications. It generates Flash/PROMable code for Renesas M16C and R8C MCUs. The latest version of the software is EWM16C v3.40.

What's Special?

The M16C software contains a simulator, a compiler and a debugger, thus providing an integrated development environment. It also has integration for IAR visualSTATE. The M16C compiler provides support for both 'C' and 'C++' languages and the debugger provides support for Renesas E100 and PC7051 and many other emulators. The software has updated device support and also provides up-to-date user documentation. C-SPY debugger lets the programmer do real-time debugging by providing versatile monitoring of registers, structures, call chain, locals, global variables and peripheral registers.

Users' feedback

The software is easy-to-use and provides drag-and-drop features. It allows good code optimisation for code density and the execution speed of the software is really good.

Suggestion

The evaluation version of the software

Tips from Experts on Microprocessors

Bauers says, "A newbie, at stage one, may start off with his 8086 software; but for stage two, developers could try the PICAXE chips from <http://www.rev-ed.co.uk/picaxe/>. There are good free tutorial guides on this site and the hardware is not expensive. The software is free too. There are lots of websites and forums for the PICAXE family of chips, so you can get online support. The chips are programmed in a simplified version of BASIC. This is excellent for beginners and you get your hands on real hardware." He adds, "For stage three, and once you have mastered this style of programming, visit microchip.com and dive into the deep end."

Masharib says, "Embedded programmers should have good knowledge of MCU architecture. They should never try and write from scratch, rather use an example and modify and build upon it. Modularity is important—ideally have a software design specification in the form of DFDs, state machines, etc and try to keep the code size small as the MCU's on-board flash is limited and the external memory is expensive and slow. Further, while programming, use optimise variables for both global and local, as each variable declaration needs MCU resources and has bearing on the performance."

Carlson says, "There are various books available on amazon.com. Some well respected authors are Keith Curtis, Lucio Di Jasio, Fred Eady, Priyabrata Sinha, Chuck Hellebuyck and John Peatman."